

어딴세이 프로젝트 학습과 복습

Flask · 데이터베이스 · 로그인 · API · CSS · Git

현재 코드를 따라 읽고, 원리를 설명하고, 직접 실습하는 한국어 교재입니다.

아이폰 열람용 / 2026-10-02

1. 학습 교재
2. 실습과 복습 - 정답 포함
3. 실제 코드 지도
4. 메모리 DB 실습 코드

표는 작은 화면에서 읽기 쉬운 항목별 형식으로 변환했습니다. PC 전용 파일 링크는 위치 안내 텍스트로 바꾸었으며, 실습 코드는 PC에서 실행합니다. 원문에 접혀 있던 정답은 모두 펼쳤습니다.

어딴세이 코드로 배우는 Flask - 학습 교재

2026-10-02의 현재 작업 폴더를 기준으로 작성했습니다. 기준 커밋은 `fa1c206`이며, 아직 커밋하지 않은 내 여행지 화면의 CSS·템플릿 수정도 포함합니다. 코드가 바뀌면 이 문서의 설명과 줄 번호도 다시 확인하세요.

공부 순서

1. 이 교재로 요청 한 번이 처리되는 흐름을 이해합니다.
2. 실제 코드 찾아보기에서 함수와 라우트를 찾습니다.
3. 실습·복습 문제를 정답을 보지 않고 풉니다.
4. 실행 가능한 실습을 읽고 실행해 ORM 결과를 관찰합니다.

하루 40~60분씩 7회에 나누는 것을 권합니다. 1회는 앱 구조, 2회는 조회·템플릿, 3회는 ORM, 4회는 로그인, 5회는 API·저장, 6회는 CSS, 7회는 Git·종합 복습입니다. 목표는 모든 파일을 외우는 것이 아니라, 화면에서 출발해 관련 코드까지 찾아가는 것입니다.

1. 이 프로젝트가 하는 일

사용자가 가입·로그인하고 여행지를 등록합니다. 여행지는 지도와 목록에 나타나고, 상세 화면에는 찜·리뷰·여행톡이 연결됩니다. 커뮤니티에는 별도의 게시글과 댓글·좋아요가 있습니다. 마이페이지의 내 여행지 목록은 로그인한 사람이 작성한 여행지만 보여줍니다.

역할: 앱 조립

실제 위치: `odysay/__init__.py`

읽을 때 할 질문: 어떤 Blueprint가 실제 등록됐나?

역할: 설정

실제 위치: `config.py`

읽을 때 할 질문: DB 주소와 설정값은 어디서 오나?

역할: 데이터 구조

실제 위치: `odysay/models.py`

읽을 때 할 질문: 어떤 컬럼이 다른 테이블을 가리키나?

역할: 요청 처리

실제 위치: `odysay/views/`

읽을 때 할 질문: URL과 함수는 어떻게 연결됐나?

역할: 입력 검증

실제 위치: `odysay/forms.py`

읽을 때 할 질문: 비어 있는 값과 중복 값은 어디서 걸러지나?

역할: HTML 생성

실제 위치: `odysay/templates/`

읽을 때 할 질문: 어떤 변수를 서버가 전달하나?

역할: 모양

실제 위치: `odysay/static/css/`

읽을 때 할 질문: HTML의 class와 선택자가 일치하나?

역할: 브라우저 동작

실제 위치: `odysay/static/js/`

읽을 때 할 질문: 어떤 URL에 fetch를 보내나?

`views`의 `a0`, `a1` 같은 파일명은 팀의 정리 방식입니다. Flask가 숫자 순서로 자동 실행하지 않습니다. 파일에 Blueprint가 있어도 앱에 등록되지 않으면 라우트가 활성화되지 않습니다. 예를 들어 `place_views.py`는 폴더에 있지만 현재 앱 팩토리의 등록 목록에는 없습니다.

2. 주소창에서 화면까지

내 여행지 페이지를 예로 들면 다음 순서입니다.

GET /homepage/mvpage/places

- Flask가 등록된 URL 규칙을 찾음
- auth views.load loaded in user가 session으로 회원 조회
- a.user에 이번 요청의 회원 저장
- a5 mvpage views.mv places 실행
- TripLocationmd를 user id로 필터링
- mvpage mv places.html에 places 전달
- Jinia가 반복문을 실행하여 HTML 생성
- 브라우저가 trip list.css와 보조 CSS를 별도로 요청
- 화면 표시

브라우저는 .py 파일을 직접 실행하지 않습니다. Flask는 완성된 HTML이나 JSON을 응답합니다. CSS는 DB에서 본인 여행지를 고르는 역할을 하지 않습니다. 서버 필터가 데이터 범위를 결정하고 CSS는 그 결과의 모양을 정합니다.

Blueprint의 세 이름

a5_mypage_views.py의 Blueprint 이름은 mypage, URL 접두사는 /homepage/mypage, 함수 이름은 my_places입니다.

```
bp = Blueprint('mvpage', name='mvpage', url_prefix='/homepage/mvpage')

@bp.route('/places')
def mv_places():
    ...
```

- 요청 주소: /homepage/mypage/places
- endpoint: mypage.my_places
- 템플릿 파일: mypage_my_places.html

이 셋은 서로 다른 이름입니다. url_for('mypage.my_places')는 endpoint를 실제 주소로 바꿉니다. render_template('mypage_my_places.html')는 파일을 찾아 HTML로 렌더링합니다. 둘을 바꿔 쓰면 BuildError 또는 TemplateNotFound를 만날 수 있습니다.

앱 팩토리

`create_app(test_config=None)`는 앱을 만들어 설정을 읽고 DB-CSRF-마이그레이션-Blueprint를 연결한 뒤 반환합니다. 테스트 설정은 일반 설정 다음에 적용됩니다. 그래서 실습에서는 실제 `odysay.db` 대신 메모리 DB를 지정할 수 있습니다.

`db = SQLAlchemy(...)`는 확장 객체를 만드는 단계이고 `db.init_app(app)`는 그 객체를 앱에 연결하는 단계입니다. 모델 import 자체가 실제 DB 테이블을 생성하지는 않습니다.

3. 본인 여행지만 조회하는 ORM

현재 `my_places()`의 핵심은 다음과 같습니다.

```
places = (  
    TripLocationmd.query  
    .filter(bv(user_id=a.user.id))  
    .order_by(TripLocationmd.created_at.desc(), TripLocationmd.id.desc())  
    .all()  
)
```

표현: `TripLocationmd.query`

의미: 여행지 상세 모델에 대한 조회 시작

표현: `filter_by(user_id=...)`

의미: 작성자가 현재 회원인 행만 선택

표현: `created_at.desc()`

의미: 등록 시간이 최신인 순서

표현: `id.desc()`

의미: 등록 시간이 같은 경우 ID로 순서 결정

표현: `all()`

의미: 조회 실행 후 결과를 리스트로 가져오기

개념적으로 아래 SQL과 같습니다. 실제 생성 SQL은 DB와 ORM에 따라 다를 수 있습니다.

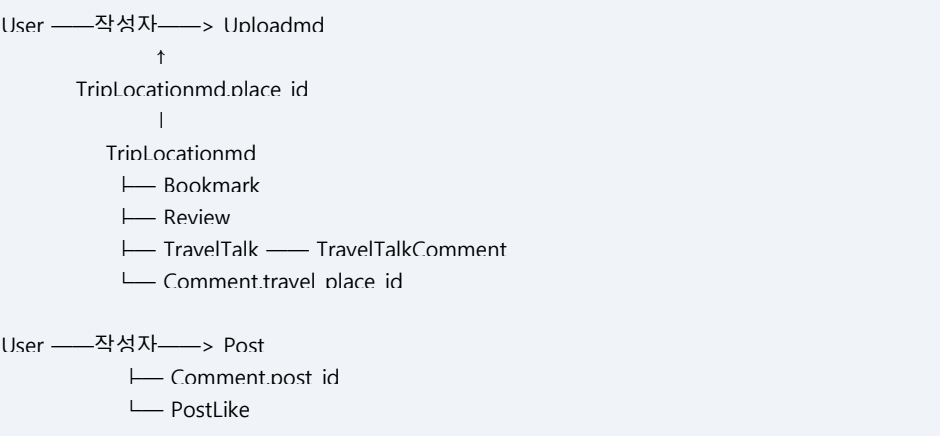
```
SELECT * FROM trip locationmd
WHERE user id = :current user id
ORDER BY created_at DESC, id DESC;
```

로그인 검사가 먼저인 이유는 `g.user`가 없을 때 `.id`에 접근할 수 없기 때문입니다. `filter_by`를 지우면 다른 사람의 여행지까지 조회됩니다. 템플릿에서 버튼만 숨기는 것으로 데이터 접근 제한을 대신할 수 없습니다.

`first()`는 첫 객체 또는 `None`, `all()`은 리스트, `count()`는 개수입니다. `get_or_404(id)`는 찾지 못하면 404 응답을 발생시킵니다. 현재 코드에는 Query 방식이 많이 사용되며, 실습은 프로젝트에 있는 표현부터 이해하도록 구성했습니다.

4. 테이블과 관계 이해하기

현재 모델의 중심 연결은 다음과 같습니다.



`Uploadmd`는 등록 정보를, `TripLocationmd`는 상세 화면 정보를 저장합니다. 두 모델에 장소 정보가 중복되어 있습니다. 이는 현재 구현의 특징이며 항상 권장되는 설계라는 뜻은 아닙니다. 한쪽만 수정하면 데이터가 달라질 수 있으므로 수정 라우트가 어느 모델을 갱신하는지 살펴보세요.

이름이 비슷한 ID 주의

- TripLocationmd.id: 상세 여행지 자체의 ID
- TripLocationmd.place_id: 연결된 uploadmd.id
- 상세 URL의 place_id: 현재 상세 뷰에서는 TripLocationmd.id를 조회하는 인자

따라서 URL 변수 이름과 DB 컬럼 이름이 같다고 같은 역할은 아닙니다. 두 테이블의 ID 값이 우연히 같아도 항상 같다고 가정하면 안 됩니다.

ForeignKey('trip_locationmd.id')는 DB 연결 규칙입니다. relationship('TripLocationmd')는 파이썬 객체 사이의 탐색 방법입니다. 현재 댓글은 comment.travel_place, 여행지는 place.comments로 연결됩니다. Comment는 일반 게시글 또는 여행지에 붙도록 두 외래키를 가지지만, 현재 모델에는 정확히 한쪽만 선택되도록 하는 CHECK 제약은 없습니다. 모델 구조와 업무 규칙을 구분해 읽어야 합니다.

Bookmark의 (user_id, place_id)와 PostLike의 (user_id, post_id)에는 고유 제약이 있습니다. 같은 사람이 같은 대상에 중복 행을 만들지 못하게 DB에서도 막습니다. 애플리케이션에서 먼저 조회하는 것과 DB 제약은 서로 보완합니다.

5. 저장: add, flush, commit, rollback

a4_upload_views.upload()의 저장 흐름은 다음과 같습니다.

```
new upload = Uploadmd(...)
db.session.add(new upload)
db.session.flush()
new trip location = TripLocationmd(place_id=new upload.id, ...)
db.session.add(new trip location)
db.session.commit()
```

add는 저장할 객체를 세션에 등록합니다. flush는 INSERT 등을 DB에 보내 자동 생성 ID를 얻지만 트랜잭션을 최종 확정하지 않습니다. commit이 저장을 확정합니다. 예외 후 rollback은 진행 중인 DB 트랜잭션을 되돌리

고 세션을 다시 쓸 수 있게 합니다.

사진 파일 저장은 별도입니다. 현재 업로드 코드는 사진을 디스크에 먼저 저장한 뒤 DB 저장을 시도합니다. DB rollback이 이미 저장된 사진까지 자동 삭제하지 않습니다. 이 차이는 실패 상황을 설계할 때 중요한 복습 포인트입니다.

등록 성공 후 상세 페이지로 redirect합니다. 브라우저는 새 GET 요청을 보내 상세 HTML을 받습니다. POST 처리 결과로 화면을 직접 반환하는 것과 이동 응답을 주는 것은 서로 다른 흐름입니다.

6. 로그인: session과 g

`auth_views.login()`은 폼 검증, 회원 조회, 비밀번호 해시 검사, 활동 상태 검사를 수행합니다. 성공하면 `session.clear()` 후 `session['user_id'] = user.id`를 저장합니다.

다음 요청에서 `load_logged_in_user()`가 세션의 ID로 회원을 찾아 `g.user`에 넣습니다.

항목: `session['user_id']`

수명과 역할: 요청 사이에 로그인 정보를 유지

항목: `g.user`

수명과 역할: 이번 요청에서 사용할 회원 객체

항목: `password_hash`

수명과 역할: 원문 비밀번호 대신 저장한 해시

Flask 기본 세션은 서명된 쿠키 방식입니다. 서명은 변조 검증용이며 암호화와 다릅니다. 세션에 비밀번호를 넣지 않습니다. `SECRET_KEY`는 세션 서명 및 CSRF 등에 중요합니다. 현재 설정 파일의 개발용 값은 배포용 비밀 키 관리와 구분해야 합니다.

인증은 '누구인가', 권한 검사는 '이 데이터를 수정해도 되는가'입니다. 로그인했다고 다른 회원의 댓글을 수정할 수 있어서는 안 됩니다. 댓글 수정 함수에서 `comment.user_id`와 현재 회원 ID를 비교하는 부분을 찾아보세요.

7. HTML 응답과 JSON 응답

내 여행지는 서버가 Jinja 반복문으로 카드를 만듭니다. 여행지 목록은 HTML을 받은 뒤 JS가 데이터를 받아 카드를 만듭니다.

내 여행지: GET → DB 조회 → render template(places=...) → 카드가 있는 HTML
여행지 목록: GET → 기본 HTML → trip_list.js → fetch('/first/places') → JSON → DOM 카드

중요한 실제 연결: 현재 trip_list.js의 loadTravelPlaces()는 /first/places를 요청합니다. /trip-list/places API도 존재하지만 이 JS가 현재 호출하는 주소는 아닙니다. 파일 이름으로 추측하지 말고 fetch 문자열을 확인하세요.

/first/places는 위도·경도가 NULL이 아닌 여행지만 조회합니다. 반면 /trip-list/places는 모든 여행지를 조회합니다. 여행지가 DB에는 있는데 지도 연동 목록에서 빠졌다면 좌표 조건도 확인해야 합니다.

JSON 객체의 키와 JS에서 읽는 속성 이름이 연결 계약입니다. 서버가 place를 보내는데 브라우저가 title만 읽으면 이름이 표시되지 않을 수 있습니다. Network 탭에서 요청 URL, 상태 코드, Response를 함께 확인하세요.

찜·리뷰·여행톡은 feature Blueprint가 처리합니다. 상세 JS는 POST 요청에 JSON과 X-CSRFToken을 함께 보내는 부분이 있습니다. 전역 CSRF 보호가 있어도 @csrf.exempt가 붙은 커뮤니티 API는 예외입니다. '현재 구현'을 '모든 API가 보호된다'로 일반화하지 마세요.

8. 템플릿과 CSS: 이번 수정 이해하기

현재 내 여행지 템플릿은 먼저 trip_list.css, 다음 mypage_my_places.css를 불러옵니다. 공통 레이아웃은 첫 파일에서 재사용하고, 내 여행지 전용 요소만 두 번째 파일에서 보완합니다.

HTML class: header, header-inner, gnb

하는 일: 상단 메뉴 구조

HTML class: travel-hero

하는 일: 제목과 소개 영역

HTML class: travel-container

하는 일: 본문 최대 너비와 여백

HTML class: destination-grid

하는 일: 카드 열 배치

HTML class: destination-card

하는 일: 카드 테두리·모서리·hover

HTML class: card-image, card-content

하는 일: 이미지와 텍스트 영역

HTML class: my-list-actions, my-place-date

하는 일: 내 여행지 전용 기능

CSS를 링크하는 것만으로 같은 화면이 되지는 않습니다. HTML class가 CSS 선택자와 일치해야 합니다. 이전 템플릿의 `travel-card`에 `destination-card` 규칙은 적용되지 않습니다.

공통 그리드는 기본 3열, 화면 너비 1000px 이하에서 2열, 520px 이하에서 1열입니다. CSS 캐스케이드는 중요도·선택자 구체성·선언 순서 등을 함께 고려합니다. 뒤에 읽은 파일이 모든 규칙을 무조건 덮어쓰는 것은 아닙니다.

공통 `.empty-state`는 JS 화면용으로 `display: none`입니다. 서버에서 빈 상태를 렌더링하는 내 여행지에는 `.my-places-page .my-empty-state { display: block; }`이 필요합니다. 같은 CSS를 재사용해도 화면 생성 방식의 차이는 보완해야 합니다.

Jinja의 `{{ ... }}`는 값 출력, `{% ... %}`는 조건·반복 같은 제어입니다. `photos`는 현재 쉼표로 구분한 파일명 문자열입니다. 템플릿이 `split` → `trim` → 빈 값 제거 → 첫 사진 선택 순서로 처리합니다. 사진이 없으면 자리 표시자를 출력합니다.

9. 오류를 찾는 순서

증상: 404

먼저 볼 곳: URL 규칙·Blueprint 등록

확인 방법: routes 출력과 주소 비교

증상: 405

먼저 볼 곳: GET/POST 허용 방식

확인 방법: Network의 Request Method

증상: BuildError

먼저 볼 곳: endpoint 오타

확인 방법: url_for 인자와 routes 비교

증상: 400, CSRF 메시지

먼저 볼 곳: 토큰 전달

확인 방법: 폼 hidden 필드·요청 헤더

증상: 로그인으로 이동

먼저 볼 곳: session, g.user

확인 방법: 302의 Location 확인

증상: no such table

먼저 볼 곳: 실제 DB 구조

확인 방법: 모델과 DB는 별개인지 확인

증상: Could not resolve relationship

먼저 볼 곳: 모델 이름

확인 방법: 이전 TravelPlace 참조 잔여 확인

증상: CSS가 안 바뀜

먼저 볼 곳: 캐시·class·파일 경로

확인 방법: 강력 새로고침·Computed 탭

증상: 목록이 비어 있음

먼저 볼 곳: user_id·좌표·API

확인 방법: SQL 필터와 JSON 확인

권장 순서는 '요청이 서버에 도착했는가 → 어떤 함수가 받았나 → 조회 결과가 있나 → 응답이 올바른가 → 브라우저가 올바르게 표현했나'입니다. 여러 층을 동시에 수정하면 원인을 확인하기 어렵습니다.

10. 현재 코드와 문서를 구별해서 읽기

기존 README는 과거 통합본의 설명이 포함되어 있어 현재 코드와 일부 다릅니다. 업로드의 실제 저장 로직은 타임스탬프 파일명을 사용하며, README의 UUID·파일 검증 설명이 지금도 구현되어 있다고 단정할 수 없습니다. 실제 업로드 뷰는 Gemini 보정, 국내 TMAP 검색, Nominatim 대체 검색 흐름이 보입니다. 별도 services 코드가 있다고 현재 뷰가 반드시 그것을 호출하는 것은 아닙니다.

외부 API의 현재 이용 가능 모델명·요금·정책은 이 교재에서 검증하지 않았 습니다. 학습은 코드에 적힌 호출 흐름을 이해하는 데 집중하며, 제공자 실 제 호출은 수행하지 않았습니다.

현재 구현에서 찾아볼 개선 과제:

- homepage.get_places()의 if p.latitude and p.longitude는 0도 좌표를 제 외할 수 있습니다. is not None과 비교해보세요.
- Uploadmd와 TripLocationmd 중복 정보 수정 시 일관성을 어떻게 유지 할까요?
- 파일 저장 성공 후 DB 실패 시 남은 파일을 어떻게 정리할까요?
- 회원마다 작성자를 다시 조회하는 반복문은 조회 횟수를 늘릴 수 있습니 다.
- 버튼 숨김과 서버 권한 검사는 다릅니다. 변경 API마다 확인하세요.

위 항목은 실습 주제이며 이 자료 작성 중에 앱 코드를 자동 수정한 것은 아닙니다.

11. Git과 DB를 따로 생각하기

Git에는 작업 파일, stage(index), commit, 원격 브랜치가 있습니다. stash는 별도 보관함입니다. DB의 스키마와 레코드는 또 다른 상태입니다.

- `git add`: 이번 커밋에 담을 내용 선택. 서버로 보내지 않음.
- `git commit`: 로컬 이력 생성. 실행 중 앱의 DB를 변경하지 않음.
- `git push`: 커밋을 원격에 전달. PR 병합과는 별개.
- `git fetch`: 원격 정보를 갱신. 현재 파일은 자동 교체하지 않음.
- `git switch`: 다른 커밋의 파일로 전환. 미커밋 변경 때문에 막힐 수 있음.
- `git stash push -u`: 수정과 미추적 파일 보관. ignored DB는 포함되지 않음.
- `.gitignore`: 이미 추적 중인 파일의 추적을 자동 해제하지 않음.

PR은 특정 소스 브랜치에서 대상 브랜치로 변경을 검토·병합하는 요청입니다. 팀 저장소의 `pr-52`를 push해도 다른 사람의 포크 `ojh`를 소스로 하는 원래 PR이 자동 갱신되지는 않습니다.

이번에 겪은 큰 ZIP 오류는 최종 폴더에서 파일을 지우는 것만으로 해결되지 않을 수 있습니다. push 대상 이력에 큰 파일이 들어 있으면 그 커밋도 정리해야 합니다. 이미 공유된 이력은 팀과 협의 없이 재작성하지 않습니다.

모델을 수정하는 것과 실제 DB에 컬럼을 추가하는 것은 다릅니다. migration은 그 차이를 옮기는 절차입니다. 이 프로젝트는 사용자가 migration을 원격에서 제외하기로 했으므로, 팀원이 코드를 pull하는 것으로 동일한 DB가 완성되지는 않습니다. 학습 실습은 별도 메모리 DB를 쓰며 실제 DB에 `upgrade/reset`을 수행하지 않습니다.

12. 복습 완료 기준

[] `/homepage/mypage/places`에서 실행되는 함수와 템플릿을 찾을 수 있다.

[] URL, endpoint, 파일명을 구분할 수 있다.

[] 내 여행지만 나오는 이유를 `filter_by`로 설명할 수 있다.

[] 상세 ID와 등록 ID를 구분할 수 있다.

[] `flush`와 `commit`, DB `rollback`과 파일 삭제를 구분할 수 있다.

[] `session`과 `g`의 차이를 설명할 수 있다.

[] 화면 생성이 Jinja 방식인지 `fetch` 방식인지 판별할 수 있다.

[] CSS 재사용에 HTML class 구조가 필요한 이유를 설명할 수 있다.

[] Git 충돌 해결과 DB 마이그레이션 해결을 구분할 수 있다.

코드를 보지 않고 위 항목을 설명한 뒤 실제 파일을 열어 답을 확인하세요.

실습과 복습 문제

교재 · 실제 코드 지도

실습 1 - 요청 경로를 손으로 추적하기 (15분)

코드를 실행하지 말고 아래 표를 채웁니다.

항목: 내 여행지 URL

항목: Blueprint 이름

항목: URL 접두사

항목: 뷰 함수 이름

항목: 로그인 확인 조건

항목: 데이터 모델

항목: 본인 데이터 필터

항목: 템플릿으로 전달하는 변수

항목: 공통 CSS와 보조 CSS

정답 / 정답과 해설

/homepage/mypage/places → mypage → /homepage/mypage → my_places → g.user is None → TripLocationmd → filter_by(user_id=g.user.id) → places → trip_list.css, mypage_my_places.css입니다. 파일명 자체가 URL을 결정하지 않고 Blueprint와 route 조합이 결정합니다.

실습 2 - 격리된 ORM 실습 (20분)

프로젝트 최상위 폴더에서 Git Bash 기준으로 실행합니다.

```
.venv/Scripts/python.exe study/practice_memory_db.py
```

PowerShell에서는 다음과 같습니다.

```
.\venv\Scripts\python.exe study/practice_memory_db.py
```

마지막 줄이 **PASS**이면 성공입니다. 실습은 메모리 DB에 회원 2명과 여행지 2개를 만들고, 내 여행지는 1개만 보이는지 검사합니다. 실제 DB·사진·원격 저장소는 변경하지 않습니다. 별도 Flask 서버를 켤 필요도 없습니다.

실행 전에 예상해보세요: flush 후 ID는 있는가? rollback 후 intro는 무엇인가? 비로그인 요청의 상태 코드는 무엇인가?

실습 파일에서 **filter_by**를 제거하면 무엇이 실패할지 예측해보세요. 앱 원본 대신 실습 파일 사본에서 바꿔봅니다. 로그인 세션을 직접 넣는 부분은 화면 테스트를 위한 것이며 실제 비밀번호 로그인 테스트는 아닙니다.

정답 / 정답과 해설

flush 후에는 생성된 ID를 사용할 수 있습니다. rollback 후 소개는 커밋했던 **A learning example**입니다. 비로그인은 302, 세션을 설정한 요청은 200입니다. 필터를 제거하면 두 여행지를 가져오므로 `len(mine) == 1` 검사가 실패합니다. 회원 정보가 메모리에 있으므로 서버를 다시 실행해 실제 로그인 화면에서 이 계정으로 로그인할 수는 없습니다.

실습 3 - CSS 적용 원리 관찰 (15분)

로그인 후 내 여행지 페이지에서 개발자 도구 Elements를 엽니다. **destination-grid**를 선택하고 Computed에서 **grid-template-columns**를 확인합니다. 화면 너비를 1200px → 900px → 480px로 바꿔봅니다. 카드가 없는 경우에도 그리드가 렌더링되지 않는다는 조건을 먼저 확인하세요.

카드가 있다면 개발자 도구에서만 `.destination-card` class를 잠시 제거해 보세요. 파일을 저장하지 않아도 테두리와 hover가 바뀌는지 볼 수 있습니다. 새로고침하면 실험이 초기화됩니다.

정답 / 예상 결과

공통 CSS 기준으로 3열 → 2열 → 1열입니다. `.destination-card` 규칙은 해당 class를 가진 요소에만 적용됩니다. 자식의 `.card-image` 같은 class는 그 대로라 이미지 스타일 일부는 남을 수 있습니다. CSS 파일 하나가 페이지 전체의 모든 모양을 일괄 결정하는 것은 아닙니다.

실습 4 - API 추적 (15분)

`/homepage/trip_list`를 열고 Network에서 Fetch/XHR를 선택한 뒤 새로고침합니다. `/first/places` 요청을 찾아 Response의 필드를 확인합니다. 코드 지도의 endpoint 표에서 담당 함수도 찾아보세요.

`/trip-list/places` 응답과 비교할 때 결과가 같아도 두 쿼리가 같은 것은 아닙니다. 현재 DB에 좌표 없는 여행지가 없다면 필터 차이가 겉으로 드러나지 않을 수 있습니다. 검증을 위해 실제 DB에 데이터를 무작정 추가하지 마세요.

정답 / 정답과 해설

`trip_list.js` → `/first/places` → `a0_mapmain_views.places` → 좌표가 NULL이 아닌 `TripLocationmd` 조회입니다. `/trip-list/places`는 `a2_trip_list_views.places`가 담당하며 좌표 유무와 무관하게 조회합니다. 응답 데이터와 UI가 다르면 JSON 필드와 JS 카드 생성 코드도 비교해야 합니다.

스스로 답하는 12문제

1. `url_for('mypage.my_places')`와 `render_template('mypage_my_places.html')`는 각각 무엇을 반환하는가?
2. 파일에 Blueprint가 선언되어 있는데 404가 나는 이유 중 하나는?

3. `g.user`와 `session['user_id']`의 차이는?
4. `TripLocationmd.place_id`와 상세 라우트의 `place_id`는 같은 ID인가?
5. 사진 저장 후 DB commit 실패 시 rollback만 하면 사진도 사라지는가?
6. 외래키와 relationship은 각각 어느 층의 연결인가?
7. 코드에서 모델 이름을 바꾸면 실제 DB 테이블도 자동 변경되는가?
8. 커뮤니티 좋아요와 여행지 찜은 같은 테이블을 쓰는가?
9. 두 CSS 파일 중 뒤에 로드한 파일이 언제나 우선하는가?
10. 충돌 표시를 지웠는데 Git에서 UU가 남는 이유는?
11. git push에 성공하면 PR도 자동으로 병합되는가?
12. `.gitignore`에 DB 파일명을 넣으면 이미 커밋한 DB도 이력에서 사라지는가?

정답 / 정답 확인

1. URL 문자열 / 렌더링된 HTML입니다.
2. 앱 팩토리에서 등록되지 않았거나 요청 URL이 접두사와 다를 수 있습니다.
3. 이번 요청의 회원 객체 / 요청 사이에 유지되는 회원 ID입니다.
4. 아닙니다. 전자는 Uploadmd ID, 현재 상세 URL은 TripLocationmd ID를 조회합니다.
5. 아닙니다. DB 트랜잭션과 파일시스템 작업은 별개입니다.
6. DB 참조 규칙 / 파이썬 객체 탐색 관계입니다.
7. 아닙니다. DB 스키마 변경과 데이터 이관이 별도로 필요합니다.
8. 아닙니다. PostLike와 Bookmark가 분리되어 있습니다.
9. 아닙니다. 선택자 구체성 등 캐스케이드 규칙을 함께 봅니다.

10. index에 해결 완료가 아직 등록되지 않았을 수 있습니다. 파일 확인 후 git add가 필요합니다.
11. 아닙니다. push는 원격 브랜치 업데이트이며 PR 검토·병합은 별도입니다.
12. 아닙니다. ignore는 이미 추적된 파일이나 과거 이력을 지우지 않습니다.

오류 상황 훈련

A. 내 여행지에서 다른 사람의 카드도 보인다

예상 원인과 확인 파일을 적으세요.

정답 / 해설

먼저 `a5_mypage_views.my_places`의 `user_id` 필터와 로그인 사용자 확인을 봅니다. CSS로 가려서 해결할 문제가 아닙니다. 쿼리 결과에 타인 데이터가 포함됐는지 격리 DB로 재현하세요.

B. 커뮤니티 상세는 열리지만 댓글 POST가 실패한다

정답 / 해설

GET 성공은 POST 경로의 성공을 보장하지 않습니다. 요청 URL·메서드, 로그인, JSON content, 응답 코드와 본문, 댓글 테이블·외래키를 차례로 확인합니다. 400·401·403·500은 서로 다른 원인일 수 있습니다. 전역 CSRF만 보고 단정하지 말고 해당 함수의 `exempt` 여부도 확인합니다.

C. 최신 main으로 이동했는데 DB 오류가 난다

정답 / 해설

Git 브랜치 전환은 ignored DB를 새 모델 구조로 변환하지 않습니다. 코드의 모델과 실제 DB schema가 일치하는지 확인합니다. DB를 삭제하거나 stamp로 오류를 숨기기 전에 현재 DB 백업과 migration 이력을 확인해야 합니다.

학습 기록

마지막 종합 과제: '사용자가 여행지를 등록한 뒤 내 여행지 화면에서 확인하는 과정'을 HTTP 요청, Flask 함수, DB 테이블, 템플릿, CSS를 모두 포함해 10문장으로 설명하세요. 교재를 닫고 설명한 뒤 코드 지도로 검증하세요.

실제 코드 지도

2026-10-02 현재 앱의 등록된 URL 규칙과 소스에서 추출했습니다.
GET/POST 표에서 자동 HEAD·OPTIONS는 생략했습니다. 줄 번호는 자료
작성 시점 기준입니다.

학습 교재 · 실습·복습

등록된 라우트

URL: /

메서드: GET

endpoint: main.index

소스: main_views.py:7

URL: /auth/check-nickname/

메서드: GET

endpoint: auth.check_nickname

소스: auth_views.py:120

URL: /auth/login/

메서드: GET, POST

endpoint: auth.login

소스: auth_views.py:75

URL: /auth/logout/

메서드: GET

endpoint: auth.logout

소스: auth_views.py:114

URL: /auth/signup/

메서드: GET, POST

endpoint: auth.signup

소스: auth_views.py:42

URL: /auth/withdraw/

메서드: POST

endpoint: auth.withdraw

소스: auth_views.py:146

URL: /first/map

메서드: GET

endpoint: first._map

소스: a0_mapmain_views.py:9

URL: /first/places

메서드: GET

endpoint: first.places

소스: a0_mapmain_views.py:15

URL: /homepage/api/places

메서드: GET

endpoint: homepage.get_places

소스: a1_homepage_views.py:38

URL: /homepage/community

메서드: GET

endpoint: community.community_all

소스: a3_community_views.py:8

URL: /homepage/community

메서드: GET

endpoint: homepage.community

소스: a1_homepage_views.py:64

URL: /homepage/community/

메서드: GET

endpoint: community.community_all

소스: a3_community_views.py:8

URL: /homepage/community/all

메서드: GET

endpoint: community.community_all

소스: a3_community_views.py:8

URL: /homepage/community/api/places

메서드: GET

endpoint: community.get_places

소스: a3_community_views.py:39

URL: /homepage/community/check

메서드: GET

endpoint: community.community_check

소스: a3_community_views.py:30

URL: /homepage/community/detail/<string:post_type>/<int:item_id>

메서드: GET

endpoint: detail.detail

소스: a3_z2_community_detail_views.py:31

URL: /homepage/community/detail/api/comment/<int:comment_id>

메서드: POST, PUT

endpoint: detail.update_comment

소스: a3_z2_community_detail_views.py:183

URL: /homepage/community/detail/api/comment/<int:comment_id>

메서드: DELETE

endpoint: detail.delete_comment

소스: a3_z2_community_detail_views.py:213

URL: /homepage/community/detail/api/comment/<string:post_type>/<int:item_id>

메서드: POST

endpoint: detail.add_comment

소스: a3_z2_community_detail_views.py:131

URL: /homepage/community/detail/api/comment/delete/<int:comment_id>

메서드: DELETE, POST

endpoint: detail.delete_comment

소스: a3_z2_community_detail_views.py:213

URL: /homepage/community/detail/api/like/<string:post_type>/<int:item_id>

메서드: POST

endpoint: detail.toggle_like

소스: a3_z2_community_detail_views.py:92

URL: /homepage/community/detail/api/post/delete/<string:post_type>/<int:item_id>
>

메서드: DELETE, POST

endpoint: detail.delete_post

소스: a3_z2_community_detail_views.py:238

URL: /homepage/community/free

메서드: GET

endpoint: community.community_free

소스: a3_community_views.py:25

URL: /homepage/community/postwrite/

메서드: GET, POST

endpoint: postwrite.write_post

소스: a3_z1_community_postwrite_views.py:27

URL: /homepage/community/review

메서드: GET

endpoint: community.community_review

소스: a3_community_views.py:15

URL: /homepage/community/tip

메서드: GET

endpoint: community.community_tip

소스: a3_community_views.py:20

URL: /homepage/main

메서드: GET

endpoint: homepage.main

소스: a1_homepage_views.py:10

URL: /homepage/map

메서드: GET

endpoint: homepage.map_page

소스: a1_homepage_views.py:33

URL: /homepage/mypage

메서드: GET

endpoint: homepage.mypage

소스: a1_homepage_views.py:73

URL: /homepage/mypage/places

메서드: GET

endpoint: mypage.my_places

소스: a5_mypage_views.py:20

URL: /homepage/mypage/settings

메서드: GET

endpoint: homepage.mypage_settings

소스: a1_homepage_views.py:88

URL: /homepage/profile_edit

메서드: GET, POST

endpoint: profile.profile_edit

소스: auth_views.py:181

URL: /homepage/sjw

메서드: GET

endpoint: homepage.homepage

소스: a1_homepage_views.py:15

URL: /homepage/trip_list

메서드: GET

endpoint: homepage.trip_list

소스: a1_homepage_views.py:59

URL: /homepage/trip_location/

메서드: GET

endpoint: trip_location.trip_location

소스: a4_z1_trip_location_views.py:118

URL: /homepage/trip_location/<int:place_id>

메서드: GET

endpoint: trip_location.trip_location_detail

소스: a4_z1_trip_location_views.py:13

URL: /homepage/trip_location/<int:place_id>/edit

메서드: GET, POST

endpoint: trip_location.trip_location_edit

소스: a4_z1_trip_location_views.py:41

URL: /homepage/trip_location/feature/bookmark/<int:place_id>

메서드: POST

endpoint: feature.bookmark_toggle

소스: a4_z1_z1_trip_location_feature_views.py:10

URL: /homepage/trip_location/feature/bookmark/<int:place_id>

메서드: GET

endpoint: feature.bookmark_status

소스: a4_z1_z1_trip_location_feature_views.py:66

URL: /homepage/trip_location/feature/review/<int:place_id>

메서드: GET

endpoint: feature.review_list

소스: a4_z1_z1_trip_location_feature_views.py:100

URL: /homepage/trip_location/feature/review/<int:place_id>

메서드: POST

endpoint: feature.review_create

소스: a4_z1_z1_trip_location_feature_views.py:133

URL: /homepage/trip_location/feature/review/delete/<int:review_id>

메서드: POST

endpoint: feature.review_delete

소스: a4_z1_z1_trip_location_feature_views.py:235

URL: /homepage/trip_location/feature/review/edit/<int:review_id>

메서드: POST

endpoint: feature.review_edit

소스: a4_z1_z1_trip_location_feature_views.py:187

URL: /homepage/trip_location/feature/travel-talk/<int:place_id>

메서드: GET

endpoint: feature.travel_talk_list

소스: a4_z1_z1_trip_location_feature_views.py:263

URL: /homepage/trip_location/feature/travel-talk/<int:place_id>

메서드: POST

endpoint: feature.travel_talk_create

소스: a4_z1_z1_trip_location_feature_views.py:305

URL: /homepage/trip_location/feature/travel-talk/comment/<int:talk_id>

메서드: GET

endpoint: feature.travel_talk_comment_list

소스: a4_z1_z1_trip_location_feature_views.py:442

URL: /homepage/trip_location/feature/travel-talk/comment/<int:talk_id>

메서드: POST

endpoint: feature.travel_talk_comment_create

소스: a4_z1_z1_trip_location_feature_views.py:483

URL: /homepage/trip_location/feature/travel-talk/comment/delete/<int:comment_id>

메서드: POST

endpoint: feature.travel_talk_comment_delete

소스: a4_z1_z1_trip_location_feature_views.py:559

URL: /homepage/trip_location/feature/travel-talk/comment/edit/<int:comment_id>

메서드: POST

endpoint: feature.travel_talk_comment_edit

소스: a4_z1_z1_trip_location_feature_views.py:520

URL: /homepage/trip_location/feature/travel-talk/delete/<int:talk_id>

메서드: POST

endpoint: feature.travel_talk_delete

소스: a4_z1_z1_trip_location_feature_views.py:410

URL: /homepage/trip_location/feature/travel-talk/edit/<int:talk_id>

메서드: POST

endpoint: feature.travel_talk_edit

소스: a4_z1_z1_trip_location_feature_views.py:363

URL: /homepage/upload

메서드: GET

endpoint: homepage.upload

소스: a1_homepage_views.py:69

URL: /homepage/upload/

메서드: GET, POST

endpoint: upload.upload

소스: a4_upload_views.py:141

URL: /trip-list/places

메서드: GET

endpoint: trip_list.places

소스: a2_trip_list_views.py:16

모델별 DB 연결

테이블명과 외래키는 현재 SQLAlchemy metadata에서 추출했습니다. 이것은 실제 DB 파일에 해당 테이블이 존재한다는 뜻은 아닙니다.

테이블: user

컬럼: id, username, email, password_hash, is_active, nickname, birth_date, gender

외래키: -

테이블: posts

컬럼: id, category, title, content, photos, created_at, likes, user_id

외래키: user_id → user.id

테이블: uploadmd

컬럼: id, country, region, place, category, intro, reason, restaurant, nearby, photos, restaurant_photos, nearby_photos, latitude, longitude, user_id, created_at

외래키: user_id → user.id

테이블: post_like

컬럼: id, user_id, post_id, created_at

외래키: post_id → posts.id, user_id → user.id

테이블: trip_locationmd

컬럼: id, place_id, country, region, place, category, intro, reason, restaurant, nearby, photos, restaurant_photos, nearby_photos, latitude, longitude, user_id, likes, created_at, updated_at

외래키: user_id → user.id, place_id → uploadmd.id

테이블: bookmarks

컬럼: id, user_id, place_id, created_at

외래키: user_id → user.id, place_id → trip_locationmd.id

테이블: comment

컬럼: id, content, author, created_at, user_id, post_id, travel_place_id

외래키: post_id → posts.id, user_id → user.id, travel_place_id → trip_locationmd.id

테이블: reviews

컬럼: id, user_id, place_id, rating, content, created_at

외래키: user_id → user.id, place_id → trip_locationmd.id

테이블: travel_talks

컬럼: id, user_id, place_id, title, content, created_at

외래키: user_id → user.id, place_id → trip_locationmd.id

테이블: travel_talk_comments

컬럼: id, user_id, travel_talk_id, content, created_at

외래키: user_id → user.id, travel_talk_id → travel_talks.id

핵심 함수를 직접 읽기

아래는 요약한 가상 코드가 아니라 현재 파일에서 발췌한 코드입니다. 긴 업로드·상세 JS는 전체 소스 링크에서 이어서 읽으세요.

my_places

odysay/views/a5_mypage_views.py:20

```

@bp.route('/places')
def mv_places():
    # 로그인한 회원만 접근 가능
    if a.user is None:
        flash('로그인이 필요합니다.')
        return redirect(url_for('homepage.main'))

    # 본인이 작성한 여행지만 최신순으로 조회
    places = (
        TripLocationmd.query
        .filter(bv(user_id=a.user.id))
        .order_by(
            TripLocationmd.created_at.desc(),
            TripLocationmd.id.desc()
        )
        .all()
    )

    return render_template(
        'mypage_mv_places.html',
        places=places
    )

```

load_logged_in_user

odysay/views/auth_views.py:13

```

@bp.before_app_request
def load_logged_in_user():
    user_id = session.get('user_id')

    # 기본값은 로그인하지 않은 상태
    a.user = None

    if user_id is not None:
        user = User.query.filter_by(id=user_id).first()

        # 없는 계정 또는 탈퇴한 계정의 로그인 해제
        if user is None or not user.is_active:
            session.clear()
        else:
            a.user = user

    # 로그인한 회원만 접근 가능한 화면

```

```

member_pages = {
    'homepage.mvpage',
    'homepage.mvpage.settings',
    'profile.profile.edit',
}

if a.user is None and request.endpoint in member_pages:
    flash('로그인이 필요합니다.')
    return redirect(url_for('homepage.main'))

```

places

odyssey/views/a0_mapmain_views.py:15

```

@bp.route('/places', methods=['GET'])
def places():
    # 위도와 경도가 모두 저장된 여행지만 조회
    travel_places = (
        TripLocationmd.query
        .filter(
            TripLocationmd.latitude.isnot(None),
            TripLocationmd.longitude.isnot(None)
        )
        .order_by(TripLocationmd.id.asc())
        .all()
    )

    result = []

    for travel_place in travel_places:
        result.append({
            'id': travel_place.id,
            'country': travel_place.country,
            'region': travel_place.region,
            'place': travel_place.place,
            'latitude': travel_place.latitude,
            'longitude': travel_place.longitude
        })

    return jsonify(result)

```

bookmark_toggle

odysay/views/a4_z1_z1_trip_location_feature_views.py:10

```
@bp.route('/bookmark/<int:place_id>', methods=['POST'])
```

```
def bookmark_toggle(place_id):
```

```
    # 로그인 확인
```

```
    if a.user is None:
```

```
        return jsonify({
```

```
            'success': False,
```

```
            'message': '로그인이 필요합니다.'
```

```
        }), 401
```

```
    # 상세페이지 여행지 확인
```

```
    TripLocationmd.query.get_or_404(place_id)
```

```
    # 현재 사용자가 이미 찜했는지 확인
```

```
    bookmark = Bookmark.query.filter_by(
```

```
        user_id=a.user.id,
```

```
        place_id=place_id
```

```
    ).first()
```

```
    # 이미 찜했다면 → 찜 취소
```

```
    if bookmark:
```

```
        db.session.delete(bookmark)
```

```
        db.session.commit()
```

```
    count = Bookmark.query.filter_by(
```

```
        place_id=place_id
```

```
    ).count()
```

```
    return jsonify({
```

```
        'success': True,
```

```
        'bookmarked': False,
```

```
        'count': count
```

```
    })
```

```
    # 찜하지 않았다면 → 찜 추가
```

```
    new_bookmark = Bookmark(
```

```
        user_id=a.user.id,
```

```
        place_id=place_id
```

```
    )
```

```
    db.session.add(new_bookmark)
```

```
db.session.commit()

count = Bookmark.query.filter by(
    place_id=place_id
).count()

return jsonify({
    'success': True,
    'bookmarked': True,
    'count': count
})
```

화면별 읽을 파일

화면/기능: 내 여행지

템플릿: mypage_my_places.html

CSS/JS: trip_list.css + mypage_my_places.css

화면/기능: 전체 여행지

템플릿: trip_list.html

CSS/JS: trip_list.css + trip_list.js

화면/기능: 여행지 상세

템플릿: trip_location.html

CSS/JS: trip_location.css + trip_location.js

화면/기능: 여행지 등록

템플릿: upload.html

CSS/JS: upload.css + upload.js

화면/기능: 커뮤니티 목록

템플릿: community.html

CSS/JS: community.css + community.js

자료 범위

.env 내용, API 키, DB 레코드, 업로드 사진, 백업 ZIP은 자료에 복사하지 않았습니다. 외부 제공자에 요청하지 않았으며 브라우저 화면과 전체 기능의 종합 검증 자료는 아닙니다.

프로젝트 README의 과거 설명보다 현재 URL 규칙:import:fetch:쿼리를 우선 확인하세요.

메모리 DB 실습 코드

PC 프로젝트 폴더의 study/practice_memory_db.py와 같은 코드입니다. PDF에서 긴 줄은 지면에 맞게 나뉘므로 실행할 때는 원본 .py 파일을 사용하세요.

```
"""Run from repository root: .venv/Scripts/python.exe study/practice_memory_db.py"""
import sys
from pathlib import Path
sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
from odysav import create_app, db
from odysav.models import User, Uploadmd, TripLocationmd, Comment

app = create_app({'TESTING': True, 'SECRET KEY': 'study-only-not-production',
                  'SQLALCHEMY_DATABASE_URI': 'sqlite:///memory.db'})
with app.app_context():
    # Only this process's memory DB is created. No production upgrade or network request.
    db.create_all()
    owner = User(username='learner', email='learner@example.invalid', password_hash='not-a-login-password')
    other = User(username='other', email='other@example.invalid', password_hash='not-a-login-password')
    db.session.add_all([owner, other])
    db.session.flush()
    print('1. flush assigned user IDs:', owner.id, other.id)
    for user, title in [(owner, 'My first place'), (other, 'Another user place')]:
        values = dict(country='Korea', region='Seoul', place=title, category='study',
                      intro='A learning example', reason='Practice', user_id=user.id)
        upload = Uploadmd(**values)
        db.session.add(upload)
        db.session.flush()
        db.session.add(TripLocationmd(place_id=upload.id, **values))
    db.session.commit()
    mine = TripLocationmd.query.filter_by(user_id=owner.id).all()
    assert len(mine) == 1
    print('2. total places:', TripLocationmd.query.count(), '/ mine:', len(mine))
    note = Comment(content='Test comment', author='Learner', user_id=owner.id, travel_place_id=mine[0].id)
    db.session.add(note)
    db.session.commit()
    assert len(mine[0].comments) == 1
```

```

print('3. relationship: place.comments contains'. len(mine[0].comments). 'comment')
mine[0].intro = 'Unsaved edit'
db.session.flush()
db.session.rollback()
assert mine[0].intro == 'A learning example'
print('4. rollback restored intro:', mine[0].intro)
client = app.test_client()
response = client.get('/homepage/mvpage/places')
assert response.status_code == 302
print('5. anonymous GET:', response.status_code)
# Set the session only to isolate page rendering, not to test password validation.
with client.session_transaction() as session:
    session['user_id'] = owner.id
response = client.get('/homepage/mvpage/places')
assert response.status_code == 200
assert 'My first place' in response.text and 'Another user place' not in response.text
assert 'css/trip_list.css' in response.text
print('6. signed-in GET:', response.status_code, '/ only owner card rendered')
print('PASS: original DB unchanged; memory data disappears when this process ends.')

```